

CM Track 17: Auth, Autorisierung und Session-Management

Datum: 2026-08-28 **Scope:** Alle `/api/*`-Routen, NextAuth-Login-Kette, Middleware-RBAC, Session-Revalidierung, 2FA, Admin-Endpunkte, IDOR-Pruefung **Methode:** Statische Analyse aller Route-Handler + Gegenproben als Tests **Teststand:** 1270 bestehend, 25 neu, 1295 gesamt, 0 Fehler, Typecheck 0

Zusammenfassung

Sechs Befunde, alle behoben. Einer davon (Empfehlungen-IDOR) erlaubte einem Anbieter, Empfehlungen in den Feed beliebiger Kunden einzupflanzen. Zwei weitere (Passwort-Aenderung ohne altes Passwort, 2FA-Attrappe) betrafen die Login-Sicherheit. Die restlichen drei (Admin-Route ohne Zod, Wait-List mit Roh-IPs, passwordMustChange nicht revalidiert) waren Luecken in der Eingabevalidierung und DSGVO-Konformitaet.

Befund 1: Passwort-Aenderung ohne altes Passwort

WAS: `POST /api/auth/change-password` akzeptierte ein neues Passwort ohne das aktuelle zu pruefen.

WARUM GEFAEHRlich: Ein gestohlenes Session-Cookie (XSS, physischer Zugriff, Session-Hijacking) genuegt, um das Passwort zu aendern und den echten Inhaber dauerhaft auszusperrern. Der Angreifer braucht das Passwort nicht zu kennen — nur das Cookie.

FIX: Zwei Modi eingefuehrt:

- **Erzwungener Wechsel** (`passwordMustChange` in der Session): kein aktuelles Passwort noetig, weil der Nutzer es nicht kennt (Admin-Reset, Erstpasswort).
- **Freiwilliger Wechsel** (Account-Einstellungen): `currentPassword` Pflichtfeld, geprueft ueber `signInWithPassword` — derselbe Weg wie der Login.

Datei: `src/app/api/auth/change-password/route.ts`

Befund 2: passwordMustChange nicht aus der DB revalidiert

WAS: `password_must_change` stand im JWT-Token aus dem Login und wurde bei Folge-Requests nie gegen die Datenbank geprueft.

WARUM GEFAEHRlich: Setzte der Admin das Flag nach dem Login des Nutzers, blieb es bis zum naechsten Login wirkungslos — bei einer Session-Dauer von 365 Tagen war das ein Jahr. Umgekehrt blieb ein beim Login gesetztes Flag bestehen, auch wenn der Admin es zuruecknahm.

FIX: `password_must_change` in die `loadAccountState`-Abfrage aufgenommen (`src/modules/auth/session.ts`). `getServerSession()` ueberschreibt das Token-Feld jetzt aus der DB — genau wie es die Rolle bereits tut. Der 15-Sekunden-Cache reicht fuer die Konsistenz.

Datei: `src/modules/auth/session.ts`

Befund 3: 2FA aktiviert, aber nicht durchgesetzt

WAS: Die Einrichtung ueber `/api/auth/2fa/setup` und `/verify` funktionierte. `authorizeCredentials` hat den TOTP-Code aber nie geprueft — 2FA war eine vollstaendig aufgebaute Funktion ohne Strom.

WARUM GEFAEHRlich: Der Nutzer glaubt, sein Konto sei mit 2FA geschuetzt. Tatsaechlich genuegt weiterhin Email + Passwort. Jeder Credentials-Leak fuehrt zum Kontozugriff, genau das, was 2FA verhindern soll.

FIX: 1. `authorizeCredentials` liest `user_2fa` und prueft den TOTP-Code, wenn 2FA aktiviert ist. Kein oder falscher Code = Login-Fehlschlag (kein Orakel: dieselbe generische Meldung wie bei falschem Passwort). 2. Neuer Endpunkt `POST /api/auth/2fa/status`: das Login-Formular fragt VOR dem Login, ob 2FA erforderlich ist, und blendet das Code-Feld ein. Kein Konto-Orakel: unbekannte Adressen liefern `{ required: false }`. Rate-Limited (10/min). 3. Login-Formular (`src/app/(auth)/auth/page.tsx`) um den Pre-Check erweitert.

Dateien: `src/modules/auth/auth.config.ts`, `src/modules/auth/auth.schemas.ts`, `src/app/api/auth/2fa/status/route.ts` (neu), `src/app/(auth)/auth/page.tsx`

Befund 4: Admin-Route ohne Zod-Validierung

WAS: `PATCH /api/admin` nahm beliebigen JSON-Body entgegen. `id` wurde nicht als UUID geprueft, `action` nicht gegen eine Whitelist validiert, `data` war ein offenes Objekt.

WARUM GEFAEHRlich: Ein kompromittierter Admin-Account (oder ein XSS in einer Admin-Seite) konnte beliebige Payloads an die Route senden. Ohne UUID-Pruefung auf `id` sind SQL-Injection-artige Angriffe auf PostgREST-Ebene denkbar. Ohne Action-Whitelist sind unbeabsichtigte Zustaende erreichbar.

FIX: `adminPatchSchema` mit:

- `action: z.enum(...)` gegen die vier geltenden Aktionen
- `id: z.string().regex(UUID)` — nur UUID-v4
- `data: z.record(z.string(), z.unknown())` — offen, aber strukturiert
- JSON-Parse-Fehler wird abgefangen (war vorher ein 500er)

Datei: `src/app/api/admin/route.ts`

Befund 5: Wait-List speicherte rohe IPs

WAS: `POST /api/wait-list` speicherte die IP-Adresse des Absenders im Klartext in der Spalte `ip`. Alle anderen Stellen im Projekt nutzten bereits `hashIp()`.

WARUM GEFAEHRlich: IP-Adressen sind personenbezogene Daten (DSGVO Art. 4 Nr. 1, EuGH C-582/14). Die Roh-Speicherung war DSGVO-widrig und inkonsequent mit dem restlichen Code.

FIX: `import { hashIp } from '@lib/ip-hash'` und Aufruf auf den Roh-Wert. Der SHA-256-Hash reicht fuer Rate-Limiting und laesst keine Rueckrechnung zu.

Datei: `src/app/api/wait-list/route.ts`

Befund 6: Empfehlungen — IDOR ueber customerId

WAS: `POST /api/recommendations` nahm `customerId` aus dem Request-Body. Ein Anbieter konnte eine beliebige User-ID einsetzen und damit Empfehlungen in den Feed eines Kunden einpflanzen, der nie bei ihm war. Ebenso wurde `salonId` nicht gegen den Aufrufer geprueft — ein Anbieter konnte im Namen eines fremden Salons empfehlen.

WARUM GEFAEHRlich: Ein boesartiger Anbieter haette:

- Produktempfehlungen an Kunden der Konkurrenz schicken koennen (Manipulation)
- Im Namen eines anderen Salons empfehlen koennen (Identitaetsdiebstahl)
- Die Buchungs-ID konnte frei gewaehlt werden — keine Zugehoerigkeit geprueft

FIX: 1. `customerId` kommt NICHT mehr aus dem Request-Body. Die Route laedt die Buchung, leitet `customerId` und `salonId` daraus ab. 2. Der Anbieter muss Inhaber des Salons sein, dem die Buchung gehoert (`salons!inner(owner_id)` gegen `session.user.id`). Admins duerfen weiterhin fuer jeden Salon empfehlen. 3. Zod-Schema mit UUID-Pruefung auf `bookingId` und `productId`. 4. GET-Pfad und View-Pfad waren bereits korrekt auf `session.user.id` beschraenkt.

Datei: `src/app/api/recommendations/route.ts`

Ohne Befund (explizit geprueft)

Bereich	Ergebnis
Cron-Routen (<code>rental-payouts</code> , <code>hard-delete</code> , <code>publish-reviews</code>)	Alle hinter <code>isAuthorizedCron()</code> mit timing-safe <code>CRON_SECRET</code> -Vergleich. Ohne Secret deaktiviert.
Demo-Konten	Doppelt gesichert: <code>NODE_ENV === 'development'</code> UND <code>!process.env.VERCEL</code> . Auf Vercel ist <code>DEMO_ACCOUNTS</code> leer.
Session-Revalidierung (<code>getSession</code>)	Rolle kommt aus der DB (15s Cache), nicht aus dem JWT. Fail-closed: DB-Fehler = keine Session. <code>invalidateAccountState()</code> fuer sofortige Cache-Entwertung.
Provider-Isolation	Salon wird ueber <code>getOwnedSalon(session.user.id)</code> abgeleitet, nie aus dem Request.
Stripe Connect	<code>provider_user_id</code> entsteht im Webhook aus dem DB-Join, nicht aus Request-Metadaten. PGRST116-Schutz seit Track 16.
Account-Loeschung (<code>/api/account/delete</code>)	Erfordert E-Mail-Bestaetigung, Race-geschuetzt.
Favorites/Notifications	Auf <code>session.user.id</code> beschraenkt.
Setup/Promote-Admin	Timing-safe, 24-Char-Minimum, Rate-Limited.
Forgot-Password	Rate-Limited, kein User-Orakel.
Register-Provider	Zod-validiert, Rate-Limited, Cleanup bei Fehler seit Track 13.
Middleware-RBAC	Korrekte Hierarchie, Owner-Pfade erlauben Provider (gewollt).
Cookie-Konfiguration	<code>httpOnly</code> , <code>sameSite: lax</code> , <code>secure</code> in Production.
Stripe Checkout	Ownership-Checks auf allen vier Branches. Preis aus DB, nie aus Request.

Testabdeckung

Neue Testdatei: `src/__tests__/e2e/auth-haertung.test.ts`

Befund	Tests	Beschreibung
1 — Passwort	6	Ohne/mit <code>currentPassword</code> , falsches Passwort, erzwungener Wechsel, zu kurz, unauthentifiziert
3 — 2FA-Status	5	Unbekannte Adresse, ohne 2FA, mit 2FA, Rate-Limit, fehlerhafter Body
4 — Admin Zod	8	UUID-Pruefung, Action-Whitelist, Status-Whitelist, Nicht-Admin, kaputter JSON, Admin-Rollen-Vergabe, <code>super_admin</code> ok, Boolean-Pruefung
5 — Wait-List IP	1	Keine Roh-IP gespeichert
6 — Empfehlungen	5	<code>customerId</code> nicht aus Body, fremder Salon abgewiesen, Nicht-Anbieter, GET nur eigene, unauthentifiziert

Gesamt: 25 neue Tests, 1295 insgesamt, alle gruen.

Geaenderte Dateien

Datei	Aenderung
src/app/api/auth/change-password/route.ts	currentPassword-Pruefung fuer freiwillige Aenderungen
src/modules/auth/session.ts	password_must_change in loadAccountState und getServerSession
src/modules/auth/auth.config.ts	2FA-Pruefung in authorizeCredentials, code -Feld
src/modules/auth/auth.schemas.ts	code in loginSchema
src/app/api/auth/2fa/status/route.ts	Neu: Pre-Login 2FA-Status (kein Orakel)
src/app/(auth)/auth/page.tsx	2FA Pre-Check im Login-Formular
src/app/api/admin/route.ts	Zod-Schema mit UUID und Action-Whitelist
src/app/api/wait-list/route.ts	hashIp() statt roher IP
src/app/api/recommendations/route.ts	IDOR behoben: customerId aus Booking, Salon-Ownership
src/__tests__/e2e/auth-haertung.test.ts	25 neue Tests